ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 8

Тема: Разработка программ с поддержкой технологии CUDA с использованием компилятора командной строки

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Ссылка на репозиторий с кодом:

https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab08

2026

Лабораторная работа № 8.1

Разработка программ с поддержкой CUDA с использованием компилятора командной строки

Цель работы

Познакомиться с особенностями разработки программ с использованием технологии CUDA.

Задание

Разработать программу, реализующую:

- CUDA-ядро для поэлементного умножения векторов;
- CUDA-ядро для поэлементного сложения векторов;
- заполнение векторов случайными данными;
- сравнение результатов GPU и CPU для операции сложения с выводом заключения о корректности.

Листинг программы

vector_ops.cu

```
#include <cuda.h>
#include <cuda_runtime.h>
#include "cuda_utils.hpp"

__global__ void VecMulKernel(const float* a, const float* b, float* c,
int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) c[i] = a[i] * b[i];
}

__global__ void VecAddKernel(const float* a, const float* b, float* c,
int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) c[i] = a[i] + b[i];
}

void vec_mul_cuda(const float* a, const float* b, float* c, int n) {
    int bytes = n * sizeof(float);
    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, bytes); cudaMalloc(&d_b, bytes); cudaMalloc(&d_c,
bytes);
    cudaMemcpy(d_a, a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, bytes, cudaMemcpyHostToDevice);
    dim3 threads(512, 1), blocks(n / 512, 1);
    VecMulKernel<<<blocks, threads>>>(d_a, d_b, d_c, n);
    cudaMemcpy(c, d_c, bytes, cudaMemcpyDeviceToHost);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
```

```
}

void vec_add_cuda(const float* a, const float* b, float* c, int n) {
    // аналогично vec_mul_cuda, ядро VecAddKernel
}
```

main.cpp (фрагмент)

```
int N = 1 << 20; // 1 048 576 элементов
std::vector<float> a(N), b(N), add_gpu(N), add_cpu(N), mul_gpu(N);
fill_random(a, -50.f, 50.f, gen);
fill_random(b, -50.f, 50.f, gen);
vec_mul_cuda(a.data(), b.data(), mul_gpu.data(), N);
vec_add_cuda(a.data(), b.data(), add_gpu.data(), N);
vec_add_cpu(a, b, add_cpu);
bool ok = compare_vectors(add_cpu, add_gpu);
```

Результаты работы программы

Размер векторов: 1 048 576 элементов (2^20).

Первые 20 элементов поэлементного умножения на GPU:

```
-530.769 -1009.159 879.536 -1050.698 -761.445 -1225.535 329.246 -
235.444 975.664 245.032 ...
```

Первые 20 элементов поэлементного сложения на GPU и CPU совпадают:

```
29.760 -4.376 64.586 1.534 -9.622 -15.848 43.238 -14.625 -62.762 -
50.653 ...
```

Результат сравнения: «Результаты CPU и GPU совпадают.»

Выводы

В ходе работы разработаны два CUDA-ядра - для поэлементного умножения и сложения векторов. Программа корректно передаёт данные между оперативной памятью и памятью GPU, запускает ядра и возвращает результат. Поэлементное сложение, выполненное на GPU, дало результаты, идентичные результатам CPU-реализации (проверка с допуском 1e-4), что подтверждает корректность вычислений на видеокарте.

Лабораторная работа № 8.2

Определение параметров видеокарты с поддержкой технологии CUDA

Цель работы

Познакомиться с особенностями интеграции инструментария CUDA в среде разработки, научиться определять основные параметры видеокарт, влияющие на скорость вычислений.

Задание

Определить число видеокарт с поддержкой CUDA и следующие параметры видеокарты: наименование, объём глобальной памяти, константной памяти, разделяемой памяти блока, число регистров в блоке, размер WARP, максимальное число потоков в блоке, версию вычислительных возможностей, число потоковых мультипроцессоров, тактовую частоту ядра и памяти, объём кэша L2, ширину шины памяти, максимальные размерности блоков и сетки.

Листинг программы

```
cudaDeviceProp dp{};
CUDA_CHECK(cudaGetDeviceCount(&device_count));
CUDA_CHECK(cudaGetDeviceProperties(&dp, i));
cout << "Name: " << dp.name << '\n';
cout << "Total global memory: " << format_bytes(dp.totalGlobalMem) <<
'\n';
cout << "Shared memory per block: " <<
format_bytes(dp.sharedMemPerBlock) << '\n';
cout << "Warp size: " << dp.warpSize << '\n';
cout << "Compute capability: " << dp.major << '.' << dp.minor << '\n';
cout << "Streaming multiprocessors: " << dp.multiProcessorCount <<
'\n';
// ... прочие поля dp ...
```

Результаты работы программы

Обнаружено 1 CUDA-устройство. Параметры GPU приведены в таблице ниже.

Параметр	Значение
Наименование	Tesla T4
Объём глобальной памяти	14.56 ГБ
Константная память	64.00 КБ
Разделяемая память блока	48.00 КБ
Регистров в блоке	65536
Размер WARP	32
Макс. потоков в блоке	1024
Версия вычислений (Compute)	7.5
Потоковых мультипроцессоров (SM)	40

Тактовая частота ядра	1590.00 МГц
Частота памяти	5001.00 МГц
Кэш L2	4.00 МБ
Ширина шины памяти	256 бит
Макс. размер блока (x×y×z)	1024 × 1024 × 64
Макс. размер сетки (x×y×z)	2147483647 × 65535 × 65535

Выводы

Используемая видеокарта Tesla T4 относится к архитектуре Turing (Compute Capability 7.5). Наличие 40 потоковых мультипроцессоров и 14.56 ГБ глобальной памяти обеспечивает высокий уровень параллелизма. Ограничение в 1024 потока на блок и 48 КБ разделяемой памяти следует учитывать при разработке CUDA-ядер. Размер WARP равен 32, что стандартно для архитектур NVIDIA.

Лабораторная работа № 8.3

Измерение пропускной способности памяти видеокарты

Цель работы

Научиться измерять пропускную способность памяти GPU при различных типах и направлениях передачи данных.

Задание

Выделить блоки памяти размером 128 МБ. Измерить пропускную способность для шести режимов копирования данных. Сравнить результаты и сделать выводы.

Листинг программы (фрагмент)

```
// Выделение обычной (pageable) памяти CPU
std::vector<unsigned char> host_src(bytes);
// Выделение page-locked (pinned) памяти CPU
cudaMallocHost(&pinned_src, bytes);
// Выделение памяти GPU
cudaMalloc(&dev_a, bytes);

// Измерение времени с CUDA-событиями
cudaEventRecord(start);
for (int i = 0; i < iterations; ++i)
    cudaMemcpy(dst, src, bytes, kind);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&ms, start, stop);
double bw = bytes / (ms / 1000.0 / iterations) / 1e9;
```

Результаты измерений

Размер буфера: 128 МБ. Число итераций усреднения: 30. GPU: Tesla T4. CPU: 2-поточный хост.

Режим копирования	Среднее время (с)	Пропускная способность (ГБ/с)	Проверка
HostToHost	0.02540	5.29	OK
HostToDevice (pageable)	0.02807	4.78	OK
DeviceToHost (pageable)	0.02791	4.81	OK
HostToDevice (pinned)	0.01085	12.37	OK
DeviceToHost (pinned)	0.01022	13.13	OK
DeviceToDevice	0.00116	115.92	OK

Выводы

Наибольшая пропускная способность достигнута при копировании в пределах глобальной памяти GPU (DeviceToDevice) - 115.92 ГБ/с, что соответствует характеристикам шины HBM2 карты Tesla T4 (теоретический пик ~320 ГБ/с).

Использование page-locked (pinned) памяти CPU увеличивает скорость обмена CPU-GPU почти в 2.6 раза по сравнению с обычной pageable-памятью (12.37 ГБ/с против 4.78 ГБ/с). Это объясняется исключением операций перекладывания данных в промежуточные буферы: pinned-страницы напрямую видны DMA-контроллеру видеокарты.

Скорость обмена между оперативной памятью и GPU ограничена пропускной способностью шины PCIe (~16 ГБ/с для PCIe 3.0 x16), что и наблюдается в результатах.

Лабораторная работа № 8.4

Оптимизация работы с глобальной и разделяемой памятью: умножение матриц

Цель работы

Научиться оптимизировать работу с глобальной памятью путём использования разделяемой памяти на примере задачи умножения квадратных матриц.

Задание

Реализовать четыре варианта CUDA-ядер умножения квадратных матриц: наивный (Naive), с кэшированием строки (RowCache), с кэшированием столбца (ColCache) и блочный с использованием разделяемой памяти (Tiled). Сравнить производительность с CPU-реализацией при размерах матриц N = 256, 512, 1024.

Листинг программы (ключевые фрагменты)

Наивное ядро (Naive):

```
__global__ void MatMulNaiveKernel(const float* a, const float* b,
                                  float* c, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= n || col >= n) return;
    float sum = 0.0f;
    for (int k = 0; k < n; ++k)
        sum += a[row*n + k] * b[k*n + col];
    c[row*n + col] = sum;
}
```

Ядро с кэшированием строки (RowCache):

```
__global__ void MatMulRowCacheKernel(...) {
    __shared__ float cached_a[1];
    // Каждый шаг k: поток 0 загружает a[row*n+k] в shared,
    // затем все потоки блока умножают на b[k*n + col]
}
```

Блочное ядро (Tiled) с использованием разделяемой памяти:

```
__global__ void MatMulTiledKernel(const float* a, const float* b,
                                  float* c, int n) {
    extern __shared__ float shared_mem[];
    float* tile_a = shared_mem;
    float* tile_b = shared_mem + blockDim.x * blockDim.y;
    // Загрузка тайлов A и B в разделяемую память блоками
tile_size×tile_size
    // __syncthreads() после загрузки
    // Вычисление частичной суммы по тайлу
}
```

Результаты измерений

Ниже приведены результаты для параметра блока/тайла param=16 (unroll=1) - штатная конфигурация из консольного вывода. CPU-результаты: N=256: 0.111 с / 0.302 GFLOPS; N=512: 1.051 с / 0.255 GFLOPS; N=1024: 13.22 с / 0.162 GFLOPS.

Алгоритм	N	Время (с)	GFLOPS	Ускорение vs CPU	Корректность
Naive	256	0.1606	0.209	0.69×	OK
RowCache	256	0.000694	48.34	160×	OK
ColCache	256	0.001775	18.90	62.6×	OK
Tiled	256	0.000543	61.74	204×	OK
Naive	512	0.002563	104.75	410×	OK
RowCache	512	0.002890	92.90	364×	OK
ColCache	512	0.011634	23.07	90.3×	OK
Tiled	512	0.001941	138.33	542×	OK
Naive	1024	0.012983	165.41	1019×	OK
RowCache	1024	0.020191	106.36	655×	OK
ColCache	1024	0.109301	19.65	121×	OK
Tiled	1024	0.007868	272.95	1681×	OK

Выводы

1. Наивная реализация (Naive) при N=256 оказалась медленнее CPU из-за накладных расходов на запуск ядра. Однако уже при N=512 она обгоняет CPU в 410 раз, а при N=1024 - в 1019 раз, что демонстрирует высокую степень параллелизма GPU при достаточном объёме данных.

2. Блочное умножение (Tiled, tile_size=16) показало наилучшие результаты во всех тестах: 204× (N=256), 542× (N=512), 1681× (N=1024). Загрузка подматриц в разделяемую память существенно снижает число обращений к медленной глобальной памяти и увеличивает эффективное использование памятной иерархии GPU.

3. Кэширование строки (RowCache) работает эффективнее кэширования столбца (ColCache) при больших N. Это объясняется нарушением условий coalescing при обращении к столбцу матрицы B, хранящейся в row-major порядке: последовательные потоки обращаются к несмежным ячейкам памяти.

4. Рекомендуется использовать блочный алгоритм (Tiled) с размером тайла 16×16 или 32×32 как оптимальную стратегию умножения матриц на GPU. При росте N преимущество тайловой оптимизации возрастает.